Кейс № 3 / Онлайн-чаевые

Приложение к презентации Tipro

Команда Tipro

Предмет защиты

Tipro показывает, как гость открывает QR с предчека, оставляет чаевые назначенному официанту и подтверждает демооплату чека вместе с чаевыми. Ресторан управляет командой и QR, а кабинеты показывают историю и показатели в пределах роли.

Команда и вклад участников

Участник	**Роль**	**Вклад в проект**
Аульченко Степан Ильич (ИУ5 МГТУ им. Н. Э. Баумана)	Разработчик	Реализация Tipro, демооплата и технические проверки.
Гордова Софья Александровна (ИУ5 МГТУ им. Н. Э. Баумана)	Аналитик	Роли, требования, модель данных, отчётность и ручные проверки ранней версии.
Зотов Артём Олегович (ИУ5 МГТУ им. Н. Э. Баумана)	Архитектор и презентатор	Схемы, потоки данных, архитектура, меры безопасности и презентация.

Содержание

1. Архитектура, данные и начисление чаевых 2–4

2. Правила QR и оплаты, ошибки, тесты, роли и отчётность 5–9

3. Журнал работы с ИИ 10

4. Подтверждающие материалы 11

Требования и границы

FR. Выпуск QR для чека, демооплата, история и отчёты по ролям.
NFR. Изоляция данных ресторанов, целостность QR и однократное начисление.
Границы. Tipro эмулирует оплату. Банковское списание и выплаты сотрудникам относятся к внешней интеграции.

Путь данных от QR до отчёта

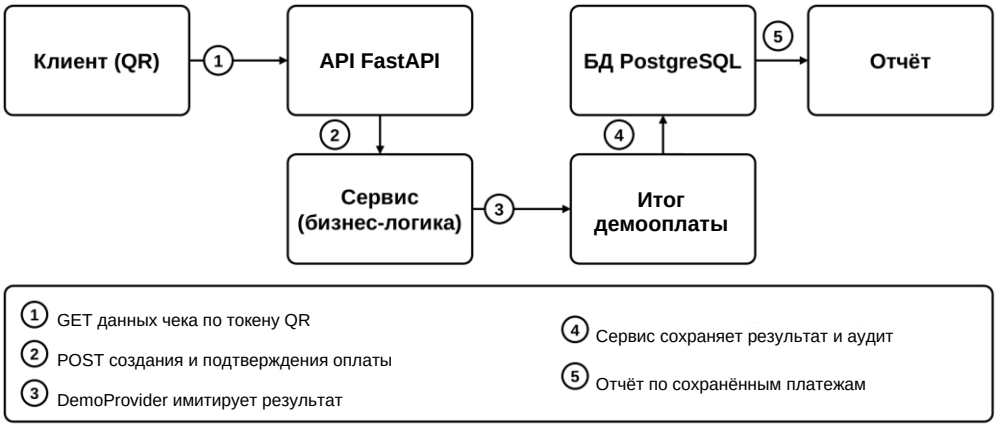


Рисунок 1. Архитектура Tipro от сканирования QR до отчёта (слайд 3).

Блок схемы	Что делает в Tipro
Клиент (QR)	Открывает предчек по QR, выбирает чаевые и подтверждает оплату.
API FastAPI	Принимает запросы браузера и вызывает сервис оплаты.
Сервис (бизнес-логика)	Проверяет QR, чек и сумму, создаёт платёж и управляет его статусом.
Итог демооплаты	Сервис проверяет результат DemoProvider в фоне или при запросе статуса. Webhook предусмотрен как отдельный путь.
БД PostgreSQL	Хранит QR, платежи, суммы чаевых и события аудита.
Отчёт	Показывает операции и сводные показатели по сотрудникам и периодам.

Клиент, API и сервис

Браузер гостя запрашивает чек через GET /api/public/restaurants/{id}?qr=… . Затем POST /api/public/qrs/{token}/payments создаёт попытку в CREATED. Отдельный POST /api/public/payments/{publicToken}/confirm запускает демооплату.

Проверка QR, запись в PostgreSQL и отчёт

QR выпускается для чека. Подпись HMAC-SHA256 покрывает ID ресторана, сотрудника и чека, nonce и exp. Сумму чека сервер берёт из связанной записи QR. Чаевые и итог проверяются отдельно.

Итог читается через GET /api/public/payments/{publicToken}. Сервер сохраняет чаевые в payments.tip и переходы в audit_events. После SUCCESS QR помечается использованным. Отчёт строится по сохранённым операциям.

В прототипе exp ограничивает сам QR. Просроченный код отклоняется, для продолжения нужен новый QR. Незавершённая оплата по истёкшему коду получает DECLINED и QR_EXPIRED.

Модель данных

PK — уникальный ключ записи. FK — ссылка на связанную запись. Имена ниже описывают логическую модель на основе серверных сущностей.

Сущность	Основные поля	Назначение
Merchant	merchant_id (PK), name, status, currency	Ресторан. Статусы PENDING, ACTIVE, BLOCKED.
Employee	employee_id (PK), merchant_id (FK), name, active, payment_details	Сотрудник и принадлежность ресторану. Реквизиты представлены маской.
QR	qr_id (PK), merchant_id и employee_id (FK), receipt_id, receipt_number, bill_amount, nonce, exp, signature, used_at	Привязка предчека и официанта. Срок действия и признак использования.
Payment	payment_id (PK), qr_id, merchant_id и employee_id (FK), amount, tip, currency, status, idempotency_key, timestamp	Попытка оплаты. amount — сумма чека, tip — чаевые.
Report	Фильтры периода, ресторана, сотрудника и валюты. Сумма, среднее, число оплат и доля успеха.	Агрегаты вычисляются по платежам.
AuditEvent	event_id (PK), payment_id, qr_id, merchant_id, action, from_status, to_status, amount_total, actor, время	След операций для разбора инцидентов.
Goal	goal_id (PK), employee_id (FK), title, target, raised, currency, active	Необязательная цель. Успешная оплата может увеличить её прогресс.

Примеры из исходных CSV

В наборе 20 ресторанов, 60 сотрудников, 100 QR и 100 платежей. Ресторан m_001 — «Вкус-Сервис Тверская», ACTIVE. Сотрудник e_001 привязан к m_001, реквизиты — ****1001. QR qr_001 связан с этим сотрудником и чеком r_1001. В записи p_0001 для m_001 и e_001 сумма чека — 637,00 ₽ , чаевые — 31,85 ₽ , статус — SUCCESS.

Связи

Ресторан 1:N Сотрудник и Ресторан 1:N QR. Сотрудник 1:N QR и Сотрудник 1:N Цель. QR 1:N Платёж для попыток до первого успеха. Платёж 1:N Событие аудита. Отчёт строится по выборке платежей.

Словарь ключевых полей

ID: целые числа (receipt_id, например 501). Чек и чаевые: NUMERIC(20,4), например 637,00 и 31,85. QR exp: Unix-время в текстовом поле. used_at: время с часовым поясом, пусто до успеха. Idempotency-Key: уникальное строковое поле платежа. В ответе API amount означает чаевые, billAmount означает сумму чека.

Начисление чаевых по webhook



Рисунок 2. Проверка webhook и начисление чаевых (слайд 4).

Шаг схемы	Что происходит
1. Webhook	Сервер получает уведомление эмулятора о результате демооплаты.
2. Подпись и id	Сервер проверяет подпись и время уведомления, отслеживает повторы по event_id и находит платёж по external_id.
3. Статус платежа	Сервер сверяет сумму и валюту, затем применяет допустимый переход статуса.
4. Чаевые начислены	При SUCCESS чаевые учитываются в отчёте, QR помечается использованным, а прогресс привязанной цели увеличивается при её наличии.

Webhook — отдельный путь обработки результата. При обычном демопоказе сервер узнаёт итог у DemoProvider через фоновую проверку или запрос статуса. Платёж проходит CREATED → PENDING → SUCCESS или DECLINED. Отмена разрешена из CREATED, а после подтверждения возвращает HTTP 409.

Режим ожидания оставляет операцию в PENDING. Если срок QR истекает до завершения, сервер завершает незакрытый платёж со статусом DECLINED и причиной QR_EXPIRED.

Проверки QR и правила оплаты

ID	Проверка и решение	Зачем
Q-1	Сверить HMAC-SHA256. При несовпадении — INVALID_QR.	Выявить подмену подписанных данных.
Q-2	Проверить exp. При истечении — QR_EXPIRED.	Ограничить срок действия 24 часами.
Q-3	Проверить used_at. После успеха повторное применение даёт QR_USED.	Исключить вторую успешную оплату по тому же QR.
Q-4	Проверить статус ресторана. Для BLOCKED — MERCHANT_BLOCKED, для PENDING — MERCHANT_PENDING.	Разрешать оплату для активного ресторана.
Q-5	Сопоставить receipt_id в запросе и записи QR. При расхождении — RECEIPT_MISMATCH.	Привязать платёж к нужному предчеку.
Q-6	Чаевые ограничены 0–50% чека. За превышение суммы — TIP_OUT_OF_RANGE.	Ограничить размер чаевых. Проверка применяется при подготовке платежа.
P-1	Сумма чека больше нуля и не превышает 1 000 000.	Проверить исходную сумму при создании QR и оплате.
P-2	Тот же Idempotency-Key и те же данные возвращают прежний платёж (200). Другая допустимая сумма чаевых с тем же ключом даёт 409.	Предотвратить дублирование одного запроса.
P-3	Итог свыше 2 000 000 отклоняется с AMOUNT_OUT_OF_RANGE.	Проверить предельную сумму. При текущих лимитах на чек и чаевые этот порог недостижим обычным вводом.
P-4	Отмена из PENDING отклоняется с CANCEL_NOT_ALLOWED.	Пользователь может отменить CREATED до подтверждения.

Подпись покрывает merchant_id, employee_id, receipt_id, nonce и exp. Шестое поле записи — signature. При генерации сервер проверяет активность ресторана и официанта, принадлежность официанта ресторану и уникальность номера чека в пределах ресторана.

Ошибки и действия пользователя

Ситуация	Код	HTTP	Можно ли повторить
Подпись неверна или QR отключён	INVALID_QR	422	С действующим QR
Срок действия QR истёк	QR_EXPIRED	410	С новым действующим QR
QR уже привёл к успешной оплате	QR_USED	409	Для новой операции нужен действующий QR
Ресторан заблокирован	MERCHANT_BLOCKED	403	После восстановления доступа
Ресторан ожидает активации	MERCHANT_PENDING	403	После активации
Чек не соответствует QR	RECEIPT_MISMATCH	422	С правильным чеком
Выбран другой сотрудник	INVALID_EMPLOYEE	422	С сотрудником, указанным в QR
Введённая сумма чаевых выше 50%	TIP_OUT_OF_RANGE	422	После исправления суммы
Итог выше максимума	AMOUNT_OUT_OF_RANGE	422	После исправления суммы
Тот же ключ и другая допустимая сумма чаевых	CONFLICT	409	Новый ключ для новой операции
По QR уже есть незавершённая попытка	QR_PAYMENT_PENDING	409	Проверить текущий платёж
Отмена после подтверждения	CANCEL_NOT_ALLOWED	409	Проверить итог операции
Эмулятор отклонил платёж	CARD_DECLINED	200[1]	Новая попытка по действующему QR
Шлюз ещё не вернул итог	PENDING[2]	200[1]	Запросить текущий статус

[1] HTTP 200 относится к получению статуса существующего платежа. Результат операции передаётся в теле ответа.

[2] PENDING — статус ожидания, поле ошибки для него не требуется. Повторяемость в таблице описывает действие клиента.

Приёмочные тесты

ID и вход	Ожидаемый результат	Факт и отметка
Т3-1 · Успех. Свежий QR, чек 637 ₽ , 5%, подтверждение.	SUCCESS. Чаевые 31,85 ₽ , итог 668,85 ₽ . В статистику и активную цель той же валюты добавляются 31,85 ₽ .	Интеграционный тест пройден. Чаевые 31,85 руб., итог 668,85 руб. Цель пополнена.
Т3-2 · Отказ. Свежий QR, чек 911 ₽ , 15%, деморежим отказа.	DECLINED, причина CARD_DECLINED. При отказе 136,65 ₽ чаевых не начисляются. Итог попытки 1 047,65 ₽ .	Интеграционный тест пройден. Режим отказа проверен на тестовой попытке в THB.
Т3-3 · Неверный QR. Изменить подписанное поле действующего QR.	INVALID_QR, HTTP 422. Платёж не создан. Для истёкшего QR отдельный ожидаемый результат — 410.	Интеграционный тест пройден. Подмена подписанного поля QR вернула HTTP 422.
Т3-4 · Повтор. Дважды отправить одинаковый запрос с тем же Idempotency-Key.	HTTP 201 при создании и 200 при повторе. Один payment_id и одна запись.	Интеграционные тесты пройдены. Повторы вернули один ID, в БД осталась одна запись.
Т3-5 · Отмена. Отменить CREATED. На втором платеже подтвердить, оставить в PENDING и запросить отмену.	Первый платёж CANCELED. Второй запрос отмены возвращает 409 CANCEL_NOT_ALLOWED. Начислений по этим действиям нет.	Интеграционные тесты пройдены. Отмена CREATED и запрет отмены PENDING подтверждены.
Т3-6 · Просрочка. Режим ожидания. Имитировать истечение QR.	DECLINED с причиной QR_EXPIRED. Поздний успех не увеличивает цель.	Интеграционные тесты пройдены. Проверены воркер и поздний webhook.

Интеграционный прогон от 09.11.2026. 36 тестов прошли за 22,43 с на отдельной тестовой базе PostgreSQL. Сценарии выше сопоставлены с автотестами. Ручная репетиция браузерного демо проводится отдельно.

Роли и подключение ресторана

Действие	Мерчант	Официант	Гость	Админ платформы
Регистрация	Да	Да	Нет	Нет
Генерация QR	Да	Нет	Нет	Нет
История в кабинете	Да	Да	Нет	Нет
Открытие QR	Да	Да	Да	Да
Оплата	Да	Да	Да	Да
Отчёты	Да	Да	Нет	Да
Активация и блокировка ресторана	Нет	Нет	Нет	Да
Управление сотрудниками	Да	Нет	Нет	Нет

Мерчант подаёт заявку, официант регистрируется по приглашению. Админ создаётся на сервере. Гостевой QR и оплата доступны без аккаунта. Мерчант видит историю своего ресторана, официант — свои платежи. Админ видит отчёты платформы и управляет ресторанами.

Регистрация мерчанта

1. Представитель вводит имя, название ресторана, email и пароль.

2. Сервер проверяет данные и уникальность email, создаёт ресторан в PENDING.

3. Администратор платформы подтверждает заявку.

4. Сервер меняет статус на ACTIVE и открывает рабочие функции.

Регистрация сотрудника

1. Представитель добавляет сотрудника и выдаёт приглашение.

2. Сотрудник указывает email и пароль.

3. Сервер проверяет приглашение, связывает аккаунт с существующей записью и помечает приглашение использованным.

Отчётность и риски

Отчёт	Метрики по выбранной валюте и периоду
По ресторану	Сумма успешных чаевых, число всех записей и успешных платежей, отказы и отмены. Доля успеха = успешные / все записи. Средний процент = среднее tip / amount × 100 по успешным платежам.
По сотруднику	Сумма успешных чаевых, число платежей, средняя сумма успешных чаевых = сумма / число успешных. Лучший день — день с максимальной суммой успешных чаевых.
По дням	Для каждой даты сумма и число успешных платежей, средний процент чаевых. Группировка учитывает выбранный часовой пояс.

Пример отчёта по payments.csv

Даты в UTC. Ресторан m_001, весь набор: 6 записей, из них успешных — 5. Чаевые — 414,85 ₽ , средние чаевые — 82,97 ₽ . Доля успеха — 83,33%, средний процент чаевых — 9,00%.

Сотрудник e_001, весь набор: 2 записей, из них успешных — 2. Чаевые — 83,60 ₽ , средние чаевые — 41,80 ₽ .

Ресторан m_001 за 01.11.2025: 3 записей, из них успешных — 3. Чаевые — 245,90 ₽ , средние чаевые — 81,97 ₽ .

Угроза	Вероятность / ущерб	Мера
Подмена QR	Средняя / высокий	HMAC-SHA256 и сверка чека и сотрудника.
Повторное использование QR	Средняя / высокий	TTL 24 ч и used_at после успеха.
Повтор одного запроса	Средняя / высокий	Idempotency-Key и блокировки при обработке.
Чтение чужих платежей	Средняя / высокий	Права проверяются на сервере по аккаунту и области доступа.
Утечка реквизитов	Низкая / высокий	Хранится маска. PAN и CVV не входят в форму.
Невозможность расследования	Средняя / средний	Аудит создания и переходов платежа.

Риски и ограничения (слайд 7). QR не содержит банковских реквизитов. Повторы запросов отслеживаются по Idempotency-Key, события webhook — по event_id. Провайдер эмулируется, данные синтетические.

Журнал работы с ИИ

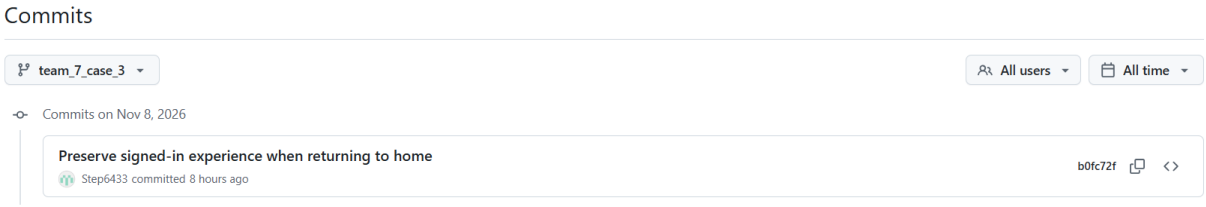
Дата	Цель	Промпт	Результат	Проверка	Использовано
02.11.26	Проверить защиту данных QR от подмены	Какие поля QR подписаны? Откуда сервер берёт сумму чека?	Сервер сверяет подпись. Чек читает из БД.	Подписаны ID ресторана, сотрудника и чека, nonce и exp. Тест на 422 меняет подписанные поля.	Да
02.11.26	Определить, где рассчитывается итог и задаётся статус	Проследи путь от PayView.vue до DemoProvider и базы. Кто вычисляет итог и кто определяет статус?	Сервис передаёт эмулятору сумму чека с чаевыми.	По services/payments.py: сервер берёт сумму из QR-записи, прибавляет чаевые и передаёт итог DemoProvider. Ответ провайдера обрабатывает transition.	Да
02.11.26	Проверить соблюдение лимита чаевых	Подбери граничные тесты на 50% чека для ручной суммы и процента. Учти округление в money и percent_tip.	Сервер проверяет предел чаевых и общий итог.	По money.py и create_payment: ручная сумма выше 50% отклоняется, процент ограничен сверху. Граничный сценарий есть в автотестах.	Да
04.11.26	Проверить обработку повторного запроса	В create_payment сравни повторы с одинаковым Idempotency-Key. Что произойдёт, если изменить сумму или получателя?	Код сравнивает QR и отпечаток запроса.	По create_payment: те же данные возвращают существующий платёж (200), новая допустимая сумма с тем же ключом даёт 409.	Да
04.11.26	Определить статус при гонке отмены и подтверждения	Что будет, если confirm и cancel придут одновременно? Разбери оба порядка захвата блокировки в public_payment.	Платёж читается с блокировкой. Отмена разрешена из CREATED.	По confirm/cancel: оба запроса блокируют платёж. Если первой прошла отмена, платёж остаётся CANCELED. Отмена из PENDING даёт 409. Гонка не запускалась.	Переработано
04.11.26	Проверить отказ при истечении срока QR	Что получит клиент, если эмулятор вернёт успех после истечения QR? Проверь путь через worker и путь через webhook.	transition проверяет срок перед записью успеха.	После срока платёж получает DECLINED с QR_EXPIRED. Проверены поздний webhook и фоновый воркер.	Да
04.11.26	Проверить изоляцию данных ресторанов	Сравни условия доступа в истории, CSV и аналитике. Можно ли подставить merchant_id другого ресторана?	Запросы ограничены по роли и ресторану.	Чужой merchant_id даёт 403. История, CSV и аналитика используют общий фильтр доступа.	Переработано
05.11.26	Проверить маскирование реквизитов в API	Проследи запись и выдачу payment_details. Достаточно ли миграции и ограничения БД, чтобы полные реквизиты не попали в API?	Миграция маскирует старые значения и добавляет CHECK.	По коду и миграции: API и БД принимают маску либо пустое значение. Есть автотест ограничений API и БД.	Переработано
05.11.26	Уточнить, какие платежи входят в расчёт доли успеха	Проверь формулу successRate. Входят ли CREATED и PENDING в знаменатель? Как изменится метрика при фильтре по завершённым оплатам?	Текущий знаменатель включает все записи выборки.	По analytics.py: успехи делятся на все записи. При одном платеже на статус расчёт даёт 1/5, а только по завершённым — 1/3.	Да
06.11.26	Зафиксировать контрольную сумму CSV	Вычисли MD5 исходного merchants.csv и добавь результат в раздел E.	Хеш показан на стр. 11.	Команда и MD5 видны на скриншоте.	Да

Формулировки промптов восстановлены ретроспективно. Отметка «Использовано» относится к выводу, который вошёл в материалы проекта.

Подтверждения

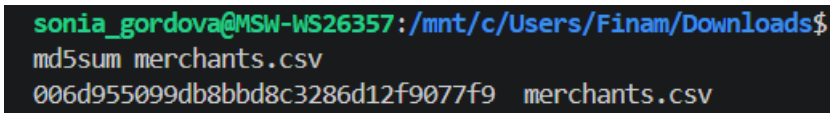
1. Ссылка на Git-ветку: https://github.com/Step6433/tipro/tree/team_7_case_3

2. Скриншот последнего коммита:



3. Интеграционная проверка Tipro: 36 тестов пройдены 09.11.2026 (см. стр. 7).

4. Контрольная сумма merchants.csv:



Хеш зафиксирован на скриншоте команды. Для повторной сверки нужен исходный файл merchants.csv.